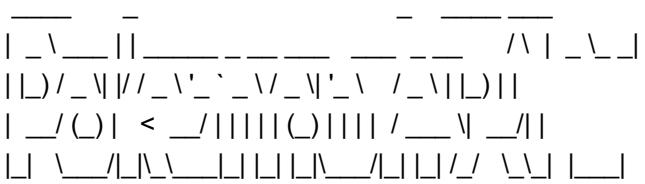
🚀 TP - 02. NodeJS - Serveur API - Pokedex

Propriétés (5)	Valeurs
version	0.9
Bloc	Bloc2
classe	BTS SIO SLAM2
Compétences du référentiel	B2.1.2 - Participer à la conception de l'architecture d'une solution applicative, B2.1.5 - Identifier, développer, utiliser ou adapter des composants logiciels, B2.2.3 - Analyser et corriger un dysfonctionnement, B2.3.2 - Développer des fonctionnalités applicatives au sein d'un système de gestion de base de données (relationnel ou non)
tags	nodejs, tp, B2, api



Mission 1: Serveur Backend en Node JS

Objectifs

- Découverte de Node JS
- Comprendre le principe des API
- Comprendre les concepts
  - client - serveur
  - backend - frontend

Vous allez devoir réaliser un véritable **Serveur API** qui fournira des données.

🛠️ Travail à faire

- Concevoir une **API REST** en NodeJS sur votre **VM distante**
- Vous devrez **régulièrement pusher** sur votre dépôt GitHub

---

Arborescence

🤳 Attention à respecter et **reproduire** l'arborescence suivante.
Tous les fichiers du serveur **backend** seront à placer dans le dossier **BACKEND**.

Les dossiers **DATA** et **FILES** et leur contenu sont à récupérer sur le NAS.
Dans le dossier **COURSNC\B2\03-Node\TP Pokedex**

```
BACKEND
├── DATA
│   ├── items.json
│   ├── moves.json
│   ├── pokedex.json
│   └── types.json
├── FILES
│   ├── images
│   ├── sprites
│   └── thumbnails
├── index.js
└── README.md
```

🛠 Travail à faire

Reproduisez l'arborescence ci-dessus
Récupérez les fichiers nécessaires au projet

Pour vous guider un peu

> ⚠ **Attention!**
>
> Cette section vous oriente dans la recherche de la documentation et des outils.
> Prenez l'habitude de chercher aussi par vous mêmes.
> Pas uniquement sur des sites d'amateurs, pas non plus seulement avec les IA.
> **Soyez curieux et restez critiques!**

Quelques outils JSON

Pour manipuler, ou afficher des fichiers JSON:

- JSON Viewer
- JSON Hero
- JSON Crack

> Mais les fichiers de ce TP sont beaucoup trop larges!

Comment lire un fichier en Node

- https://www.geeksforgeeks.org/node-js/node-js-fs-readfile-method/

Comment parser un fichier lu en JSON

- https://www.geeksforgeeks.org/javascript/javascript-json-parse-method/

Mission 2: Mise en place du projet

Dans le dossier **BACKEND**:

- Création d'un **dépôt git local**

```
git init .
```

- Installation de Node, npm et des dépendances **(si besoin)**

```
sudo apt install nodejs npm -y
# Dépendances
npm install express
```

Création du script du serveur

On crée un fichier serveur nommé *index.js*

index.js

```
/**
 *  Serveur Backend Pokedex
 */

//console.log ("Hello World!");

// Définir l'emplacement des fichiers bases de données
const POKEDEX_SRC = "./DATA/pokedex.json";

// Définir l'emplacement des images
const IMAGES_SRC = "./FILES/images";
```

```
// Définir un port
const PORT = 5001;

// ************************************************
// Lancer un serveur express sur un port défini

const fs = require('fs');

// npm install express
const express = require('express');
const app = express();

// Lancement du serveur et attendre
app.listen(
    PORT,
    '127.0.0.1',
    () => {
        console.log('Server Pokedex is listening on ' + PORT);
    }
)
```

Pour le lancer, tapez:

```
# Pour lancer plutôt en monitoring (rechargemet à chaque modification du fichier)
node --watch index.js
```

Mission 3: Routes à créer

> https://expressjs.com/en/guide/routing.html
> https://laconsole.dev/formations/express/routage

Ce que l'on appelle **route** correspond aux **urls** qui seront **appelables sur votre API**.
😊 Autrement dit, les **endpoints**.

Comment vérifier les routes que vous allez créer?

> Avec **PostMan** ou directement dans votre navigateur.

> ⚠ **Attention!**
>
> - Votre API doit contenir un **préfixe** dans ses Urls comme suit:
>   **/api/v1/**
>   où **v1** correspond à la version de votre API

{prefix}/pokemons

Pour afficher **tous les pokemons**

Algorithme:

1. Création de la route ***/pokemons***
2. Lire le contenu du **pokedex** (fichier: *DATA/pokedex.json*)
3. Renvoyer en réponse **l'intégralité du contenu** du fichier *(809 pokemons)*

> 🛠 **Travail à faire**
>
> Créer cette **route** et la fonction associée
> Implémentez la fonction correspondante

Vérifiez que vous **renvoyez** bien le fichier **JSON** dans son intégralité

Vérifiez les **Headers** de la réponse

{prefix}/pokemons/random

Pour renvoyer **un seul** pokemon au **hasard**

Algorithme:

1. Création de la route ***/pokemons/random***
2. Définition d'un numéro au hasard compris entre le **minimum** et le **maximum**
    1. Quel est l'id minimum possible?
    2. Quel est l'id maximum possible?
        1. Quelle est la taille du pokedex?
3. Renvoi du pokemon ayant l'id **hasard**

- https://www.w3schools.com/js/js_random.asp

> **🛠 Travail à faire**
>
> Créer cette **route** et la fonction associée
> Implémentez la fonction correspondante

{prefix}/pokemons/{id}

Pour renvoyer **un seul** pokemon à partir de son **identifiant** passé en paramètre *(si le pokemon correspondant existe)*

🤳 Ici, le paramètre **doit** être un nombre.

> https://laconsole.dev/formations/express/routage#paramètres-dynamiques

Algorithme:

1. Création de la route */pokemon/id*
2. Récupération du paramètre *id* envoyée dans la requête
3. Recherche du *pokemon* portant cet *id*, On peut aussi parcourir le tableau et s'arrêter à la case *id*

4. Renvoi du pokemon correspondant

🛠️ Travail à faire

Créer cette **route** et la fonction associée
Implémentez la fonction correspondante

{prefix}/pokemons/{name}

Pour renvoyer **un seul** pokemon à partir de son **nom** passé en paramètre *(si le pokemon correspondant existe)*

🤳 Ici, le paramètre **doit** être du **texte**.
Attention la route **/pokemons/hasard** a été créée plus haut!

> https://laconsole.dev/formations/express/routage#paramètres-dynamiques

Algorithme:

- A vous de le trouver!

🛠 Travail à faire

Créer cette **route** et la fonction associée
Implémentez la fonction correspondante

Rappel des routes

> **⚠ Attention!**
>
> - Pour **rappel**, toutes les routes sont **préfixées** de: **/api/v1/**

Route	Paramètre	Nom de la route	Description
/pokemons		findAll	Renvoie TOUS les pokemons
/pokemons/random		findOneByRandom	Renvoie UN pokemon au hasard
/pokemons/{id}	**id**: integer Identifiant d'un pokemon	findOneById	Renvoie UN pokemon à partir de son **id** Si non trouvé renvoie **404**

Route	Paramètre	Nom de la route	Description
/pokemons/{name}	**name**: string Nom d'un pokemon	findOneByName	Renvoie UN pokemon à partir de son **nom** Si non trouvé renvoie **404**

Mission 4: Versioning

Votre projet **doit** être versionné.

🛠 Travail à faire

- Création du dépôt distant GitHub
  - **commit** et **push** fréquents sur le dépôt distant
  - Documentation dans le fichier *README.md*
- Clonage du dépôt distant sur la VM distante

Evaluation

Consignes à respecter

- Mise à disposition des sources via GitHub (envoi du lien du dépôt sur le serveur SLAM2)
- Respect des consignes (arborescence, noms des fichiers)
- Vous **devez** à minima faire ce qui est demandé. Ensuite, lâchez vous, ajoutez autant de fonctionnalités que vous désirez!
- Un **frontend**
- **Démonstration sur la VM distante validée** par l'enseignant

Critère d'évaluation	Non acquis (ou insuffisant)	Partiellement Acquis	Acquis (ou très bien)
Code source disponible	Sources non disponibles via GitHub, commits obsolètes	Sources disponibles mais versionning mal géré	Gestion correcte des sources, commits pertinents
Respect des consignes	Consignes non respectées	Consignes partiellement respectées	Consignes totalement respectées
Réalisation	Rien n'a été réalisé, ou rien ne fonctionne	Réalisation partielle ou non fonctionnelle	Toutes les demandes sont réalisées et fonctionnelles
Frontend	Aucun frontend	Frontend partiel	Frontend qui exploite toutes les routes
Qualité du code	Pas d'indentation, pas de commentaires	Indentations et commentaires partiellement présents	code propre, indentation et commentaires présents et pertinents
Démonstration	Non fonctionnelle	Partiellement fonctionnelle	Fonctionnelle

---

Proposition de solution
